

Edge Impulse DSP Configuration

Smart Shoe Activity Classifier · ESP32-S3 + Zephyr RTOS · TFLite Micro INT8

1. Project setup

Parameter	Value
Project name	smart_shoe_activity_v1
Target device	Espressif ESP32-S3
Sample rate	200 Hz (must match firmware CONFIG)
Label set	walking, running, stairs_up, stairs_down, standing, stumble_pre

2. Data acquisition

Sensor axes (order matches ImuMeas struct in firmware)

Channel	Axis	Unit
0	accel.x	m/s ²
1	accel.y	m/s ²
2	accel.z	m/s ²
3	gyro.x	rad/s
4	gyro.y	rad/s
5	gyro.z	rad/s

Collection protocol

Label	Min duration	Collection condition
walking	5 min	Normal pace, flat surface
running	3 min	6–10 km/h treadmill
stairs_up	3 min	Standard staircase, repeated
stairs_down	3 min	Same staircase descending
standing	5 min	Weight shifts allowed
stumble_pre	50 events	500ms window BEFORE detected stumble (label retrospectively from flash log)

Tip: Use `eskf_stride_complete()` timestamps to extract per-stride windows from your LittleFS flash log. Export as CSV: `timestamp_ms`, `ax`, `ay`, `az`, `gx`, `gy`, `gz`. Train/test split: 80/20 (Edge Impulse default).

3. Impulse design

Input block

```
"input_block": {
  "type": "time_series",
  "window_size_ms": 2000,
  "window_increase_ms": 1000,
  "frequency": 200,
  "axes": ["accX", "accY", "accZ", "gyrX", "gyrY", "gyrZ"]
}
```

DSP block — Spectral Analysis

```
"type": "spectral_analysis",
"config": {
  "analysis_type": "FFT",
  "fft_length": 64,
  "noise_floor_db": -52,
  "filter_type": "low",
  "filter_cutoff": 20,
  "filter_order": 6,
  "features": [
    "rms", "zero_crossing_rate",
    "spectral_power_0_5", "spectral_power_5_10",
    "spectral_power_10_20", "spectral_skewness",
    "spectral_kurtosis", "peak_frequency"
  ],
  "do_log": true,
  "do_fft_overlap": true
}
```

4. DSP feature set — rationale

Feature	Axes	Rationale
RMS	All 6	Motion magnitude — separates standing from walking from running
Zero crossing rate	All 6	Periodicity — running ZCR > walking; standing ≈ 0
Spectral power 0–5 Hz	Accel xyz	Gait cadence band (60–180 steps/min = 1–3 Hz, harmonics to 5 Hz)
Spectral power 5–10 Hz	Accel xyz	Running impact harmonics, stair impact signature
Spectral power 10–20 Hz	Gyro xyz	Foot rotation in swing — high in stairs, low in walking
Spectral skewness	Accel Z	Asymmetry of vertical accel distribution — stairs up vs down

Spectral kurtosis	Accel Z	Peakedness — heel-strike impulse shape indicator
Peak frequency	All 6	Dominant cadence frequency per axis

Total output: 8 features x 6 axes = 48 float features per window. After log transform + INT8 quantisation: 48-byte input tensor.

5. Neural network architecture

Recommended — Dense classifier (fast, small)

```
Input: (48,) – flattened spectral features
|
Dense(64, relu)
Dropout(0.25)
Dense(32, relu)
Dropout(0.25)
Dense(6, softmax)
|
Output: (6,) – class probabilities
```

Alternative if accuracy < 92% on validation set

```
Input reshaped: (6, 8) – axes x features per axis
|
Conv1D(filters=16, kernel=3, relu)
Conv1D(filters=32, kernel=3, relu)
GlobalAveragePooling1D()
Dense(32, relu)
Dropout(0.2)
Dense(6, softmax)
|
Output: (6,)
```

INT8 quantisation (Edge Impulse Expert Mode)

```
"post_training_quantization": {
  "mode": "int8",
  "representative_dataset": "training_data",
  "input_quantization": true,
  "output_quantization": false
}
```

After quantisation, run Model Testing on your held-out set. Target: INT8 accuracy within 2% of float32 baseline. If drop > 2%: increase training cycles or reduce Dropout rate.

6. Stumble-precursor class — special handling

A. Class weighting during training

```
"class_weights": {
  "walking": 1.0,
  "running": 1.0,
  "stairs_up": 1.5,
  "stairs_down": 1.5,
  "standing": 1.0,
  "stumble_pre": 6.0 // increase to 8.0 if recall < 90%
}
```

B. Per-class threshold at inference (firmware)

```
/* activity_classifier.c */
uint8_t classify_with_stumble_guard(const float *probs) {
  /* Stumble alert fires at lower confidence than other classes */
  if (probs[CLASS_STUMBLE_PRE] > 0.55f) {
    gatt_notify_alert(ALERT_STUMBLE);
    return CLASS_STUMBLE_PRE;
  }
  uint8_t best = 0;
  for (int i = 1; i < NUM_CLASSES; i++)
    if (probs[i] > probs[best]) best = i;
  return best;
}
```

Do NOT use argmax for stumble_pre. A missed stumble warning is worse than a false alert. Plot the ROC curve and choose the threshold at recall ≥ 0.90 operating point. Tune 0.55f against your validation set.

7. Export for Zephyr TFLite Micro

Step 1: Download from Edge Impulse

Go to Deployment → TensorFlow Lite (INT8). Download ei-smart-shoe-activity-v1-arduino-1.0.x.zip (the .h files work with Zephyr despite the "arduino" label).

Step 2: Extract model files

```
unzip ei-smart-shoe-activity-v1-arduino-1.0.x.zip
cp src/model-parameters/model_variables.h smart_shoe/src/ml/
cp src/tflite-model/trained_model_compiled.h smart_shoe/src/ml/model.h
```

Step 3: Kconfig additions (prj.conf)

```
CONFIG_TENSORFLOW_LITE_MICRO=y
CONFIG_TENSORFLOW_LITE_MICRO_LOG_VERBOSITY=0
CONFIG_TENSORFLOW_LITE_MICRO_ARENA_SIZE=49152 # 48 KB
CONFIG_FPU=y
CONFIG_FPU_SHARING=y
```

8. Accuracy benchmarks — pass criteria before Phase 3

Test	Pass criterion	Verification method
Held-out accuracy (float32)	>= 94%	Edge Impulse Model Testing tab
INT8 quantised accuracy	>= 92%	float32 minus max 2%
Stumble_pre recall	>= 90%	ROC curve, validation set
Stumble_pre false positive rate	<= 1 per 10 min	Normal walking session
Inference time on ESP32-S3 INT8	<= 3ms	k_cycle_get_32() around invoke()
Flash usage (model only)	<= 150 KB	west build .map file
Tensor arena peak watermark	<= 48 KB	Fill arena 0xAA, check high mark

If stumble recall < 90%: increase class weight to 8.0 and retrain. If inference > 3ms: reduce Dense(64) to Dense(32) and retrain.

9. Retraining protocol (after 30+ new sessions)

Step	Action
1	Export new sessions from flash log as CSV
2	Upload via CLI: ei-uploader --api-key \$EI_API_KEY --label walking new_sessions/*.csv
3	Retrain with same architecture using transfer learning from existing weights
4	Validate: accuracy must not regress more than 1% from baseline
5	Export new model.h and deploy via Phase 4 OTA pipeline (MCUboot + BLE DFU)

Generated for Smart Shoe gait analysis platform — ESP32-S3 / Zephyr RTOS / BLE GATT / Flutter. Part of Phase 2 (On-Device ML) implementation.